COMPLETE SPECIFICATION
APPLICANT (S): VELLORE INSTITUTE OF TECHNOLOGY

FORM 2

THE PATENT ACT 1970

(39 of 1970)

&

The Patent Rules, 2003

COMPLETE SPECIFICATION

(See section 10 and rule 13)

1. TITLE OF THE INVENTION

METHOD AND SYSTEM FOR SECURE COMMUNICATION BETWEEN COMPUTATIONAL STORAGE DRIVES AND HOST SYSTEM

2. APPLICANT (S)

NAME	**VELLORE INSTITUTE OF TECHNOLOGY**
NATIONALITY	**INDIAN**
ADDRESS	Vellore Institute of Technology, Near Katpadi Road, Vellore, Tamil Nadu, India – 632014

3. PREAMBLE TO THE DESCRIPTION

The following specification particularly describes the invention and the manner in which it is to be performed.

COMPLETE SPECIFICATION
APPLICANT (S): VELLORE INSTITUTE OF TECHNOLOGY
4. DESCRIPTION

METHOD AND SYSTEM FOR SECURE COMMUNICATION BETWEEN COMPUTATIONAL STORAGE DRIVES AND HOST SYSTEM

TECHNICAL FIELD

[0001] The present invention relates to the near data processing systems and methods. The present invention also relates to computer storage system security systems and methods. The present invention additionally related to secured communication between storage drives and host systems. Further, the present invention specifically relates to method and system for secure communication between computational storage drives and host system.

BACKGROUND OF THE INVENTION

[0002] The research in the field of computational storage drives revolves around developing and implementing near data processing or IN-SITU architectures and different standards of interfacing of these architectures and devices with the traditional systems. Some proprietary systems have been already produced and released in the market which are still lagging behind in terms of security. The industries are still developing standardized security protocols for computational storage drives which can lead to inconsistencies between different

vendors of the computational storage drives.

[0003] In one embodiment of prior art, US20230112448A1 teaches a computational storage drive using FPGA implemented interface wherein the knowledge of the computational storage architecture with FPGA is required for deploying a security technique on it. In another embodiment of prior art, US20220057959A1 teaches computational storage systems and methods wherein the system teaches a In-Situ processing system enhanced by applying the proposed security technique to safeguard the communication and data. In an additional embodiment of prior art, JP2022524487A teaches computational data storage system teaching storage controllers and processors for processing light weight functions within storage drive rather than using Host CPUs.

[0004] Since the Computational storage drives (CSDs) are empowered with FPGA and Arm processors they have capability to execute functions or programs on their own. These capabilities of the CSDs can be utilised for executing data intensive applications and reduce the movement of bigdata between the hosts and CSDs. Programs like encryption, decryption, compressions are all executed IN-SITU where by the processing capacity in the CSD are utilized. Since CSDs can

also be directly connected to internet and accessed through IP all the types of cyber threats are also applicable to it. Based on the observation from CSDs case studies it is suggested that CSDs are going to play a major role in IoT environment or will be utilised with an Edge device for data storage. Since in IoT there is always the possibility of unauthorized access to device and data can occur provided the security measures are weak. To keep the data safe in CSDs and to protect it from unauthorized access a strong security mechanism needs to be detained that should be working on two different keys and only the authenticated nodes should able to access CSDs. At the same time the main challenge is that the security functions executing on CSDs must not over consume its processing capacity leaving the purpose CSDs invalid.

[0005] Fulfilling the near data processing objective of CSDs, there may be possibility of using traditional methods that utilize a single key for both authentication and encryption generally consume less processing power. While traditional methods offer faster performance and reduced processing power usage, they come with security trade-offs. Using a single key for both purposes makes the system more vulnerable if the key is compromised, as both

authentication and encryption are compromised at once. Finally, the problems focused by the proposed security mechanism is the lack of standardized security protocols for CSDs leading to inconsistencies between different vendors and potential security gaps. Secondly, CSDs directly connected to the internet are vulnerable to increased attack surface. Thirdly, implementing storage security mechanisms with separate keys but consuming less processing power thereby reducing the Security vs. performance trade-off.

[0006] **Problem Statement**: the conventional solutions that are provided for the securing the CSDs are at hardware level. One of the solutions is Hardware security Modules (HSMs) where CSDs are embedded with HSMs which facilitates in securing storage and execution capabilities mitigating the impact on processing power while enhancing security. But the problem of cross vendor smooth communication with these CSDs still pose a great challenge. Second type of solutions that exist is Hardware acceleration for specific algorithms. Some vendors are exploring hardware acceleration for encryption and decryption algorithms used in CSD security protocols, potentially improving performance without compromising security strength.

[0007] Based on the foregoing a need therefore exists for an improved method and system for secure communication between computational storage drives and host system, as discussed in greater detail herein.

SUMMARY OF THE INVENTION

[0008] The following summary is provided to facilitate an understanding of some of the innovative features unique to the disclosed embodiment and is not intended to be a full description.

[0009] It is, therefore, one aspect of the disclosed embodiments to provide for an improved computer storage system security system and method.

[0010] It is another aspect of the disclosed embodiments to provide for improved secured communication between storage drives and host system.

[0011] It is further aspect of the disclosed embodiments to provide improved method and system for secure communication between computational storage drives and host system.

[0012] Method and system for secure communication between computational storage drives and host system. A nonce generator for generating random nonces (S and R) at CSD and Host wherein a random nonce (S) is generated at CSD using

which an authentication token “AuthTd” and a session token “SessTd” is created and the provided keys and sent to the Host wherein the host upon receiving the tokens, extracts the nonce (S) and generates another authentication token “AuthTd'” wherein the “AuthTd'” is matched with “AuthTd” and host generates a nonce (R) and a session key “SessK” and sends a new session token “SessTh” to the CSD. Further, The CSD extracts the nonce (R) from the received session token and generates a session key “SessKd” wherein the “SessKd” is matched with “SessK” in order to establish a secured communication between the CSD and the Host system.

[0013] A hash function calculates the authentication tokens (AuthTd, AuthTd') and the session key (SessK, SessK') using a one-way hashing algorithm wherein the hashing ensures data integrity and prevents tampering wherein it takes inputs (Host ID, authentication key, nonces) and performs a mathematical operation to generate a unique and fixed-size output (hash value). An encryption engine encrypts the nonce (S) sent by the Host to create a session token (SessTd) and encrypts the CSD's nonce (R) to create a confirmation session token (SessTh) wherein the encryption protects sensitive data during transmission and uses a

shared encryption key (EnK) and a chosen encryption algorithm (like AES) to transform the nonce into an unreadable format. A decryption engine decrypts the received session tokens (SessTd, SessTh) to retrieve the original nonces wherein the decryption engine uses the shared encryption key (EnK) and the decryption algorithm to convert the encrypted session tokens back into their original nonce values. An authentication module handles authentication by generating and verifying authentication tokens (AuthTd, AuthTd') wherein it combines Host ID, authentication key (AuthK), and nonces, and feeds them to the hash function to create AuthTd (at Host) and AuthTd' (at CSD) wherein the authentication module compares them for a successful authentication check.

[0014] A session management module manages session key generation and utilizes the session key for secure communication wherein it combines various inputs (like Host ID, received tokens, retrieved nonces) and feeds them to the hash function to generate the session key (SessK, SessK') and verifies if the locally generated and calculated session keys match, indicating a successful session establishment. A communication interface enables data exchange between the Host and the CSD which is not directly involved in the secure

communication algorithm but crucial for transmitting tokens and establishing the communication channel wherein the communication interface handles sending and receiving of authentication tokens (AuthTd, AuthTd') and session tokens (SessTd, SessTh) between the Host and the CSD.

DETAILED DESCRIPTION

[0015] The values and configurations discussed in these non-limiting examples can be varied and are cited merely to illustrate at least one embodiment and are not intended to limit the scope thereof.

[0016] The embodiments now will be described more fully hereinafter with reference to the accompanying drawings, in which illustrative embodiments of the invention are shown. The embodiments disclosed herein can be embodied in many different forms and should not be construed as limited to the embodiments set forth herein; rather, these embodiments are provided so that this disclosure will be thorough and complete, and will fully convey the scope of the invention to those skilled in the art. Like numbers refer to like elements throughout. As used herein, the term "and/or" includes all combinations of one or more of the associated listed items.

[0017] Unless otherwise defined, all terms (including technical and scientific terms) used herein have the same meaning as commonly understood by one of ordinary skill in the art to which this invention belongs. It will be further understood that terms, such as those defined in commonly used dictionaries, should be interpreted as having a meaning that is consistent with their meaning in the context of the relevant art and will not be interpreted in an idealized or overly formal sense unless expressly so defined herein.

[0018] **Invention**: Method and system for secure communication between computational storage drives and host system. A nonce generator for generating random nonces (S and R) at CSD and Host wherein a random nonce (S) is generated at CSD using which an authentication token “AuthTd” and a session token “SessTd” is created and the provided keys and sent to the Host wherein the host upon receiving the tokens, extracts the nonce (S) and generates another authentication token “AuthTd"” wherein the “AuthTd"” is matched with “AuthTd” and host generates a nonce (R) and a session key “SessK” and sends a new session token “SessTh” to the CSD. Further, The CSD extracts the nonce (R) from the received session token and generates a session key “SessKd” wherein the

"SessKd" is matched with "SessK" in order to establish a secured communication between the CSD and the Host system.

[0019] A hash function calculates the authentication tokens (AuthTd, AuthTd') and the session key (SessK, SessK') using a one-way hashing algorithm wherein the hashing ensures data integrity and prevents tampering wherein it takes inputs (Host ID, authentication key, nonces) and performs a mathematical operation to generate a unique and fixed-size output (hash value). An encryption engine encrypts the nonce (S) sent by the Host to create a session token (SessTd) and encrypts the CSD's nonce (R) to create a confirmation session token (SessTh) wherein the encryption protects sensitive data during transmission and uses a shared encryption key (EnK) and a chosen encryption algorithm (like AES) to transform the nonce into an unreadable format. A decryption engine decrypts the received session tokens (SessTd, SessTh) to retrieve the original nonces wherein the decryption engine uses the shared encryption key (EnK) and the decryption algorithm to convert the encrypted session tokens back into their original nonce values. An authentication module handles authentication by generating and verifying authentication tokens (AuthTd, AuthTd') wherein it combines Host ID,

authentication key (AuthK), and nonces, and feeds them to the hash function to create AuthTd (at Host) and AuthTd' (at CSD) wherein the authentication module compares them for a successful authentication check.

[0020] A session management module manages session key generation and utilizes the session key for secure communication wherein it combines various inputs (like Host ID, received tokens, retrieved nonces) and feeds them to the hash function to generate the session key (SessK, SessK') and verifies if the locally generated and calculated session keys match, indicating a successful session establishment. A communication interface enables data exchange between the Host and the CSD which is not directly involved in the secure communication algorithm but crucial for transmitting tokens and establishing the communication channel wherein the communication interface handles sending and receiving of authentication tokens (AuthTd, AuthTd') and session tokens (SessTd, SessTh) between the Host and the CSD.

[0021] **Working of the invention:** The System that this disclosure is emphasizing is the smart computational storage drives which is capable of performing In-Situ processing of tasks. This kind of storage drives have

computational processing elements like FPGA or CPUs embedded on to them. These processing units are capable of accepting and executing functional tasks of varying application from different hosts. This type of storage drives vastly reduce the data movement which further influences the reduction in the time taken to transfer the data from the storage drive to host CPU. At the same time, the application or part of application executing on the host machine is offloaded to these drives for performing the function on the data. The environment of such execution is feasible through this CSDs. The embedded Realtime processors will take-up the task of executing such off-loaded functions. In this scenario, all the hosts must not off-load the application at the same time also the host should not be a Rouge machine which may intent to deprive the service of CSDs to other hosts. In order to protect the communication between the hosts and CSDs or between CSDs and CSDs this SafeConnectKey method is designed. CSDs are type of Solid-State Disks (SSD) that has computational capabilities built into it. This means that some data processing tasks can be performed directly on the SSD, reducing the need to transfer data back and forth between the SSD and the host CPU. This can significantly improve performance for certain types of tasks. The

In-Situ SSD includes several components which is shown in FIG.1.

[0022] The proposed method is implemented using seven components including Nonce Generator, Key Retriever, Token Generator, Session Token Generator, Communicator, Token verifier, and Session key Generator. These 7 components are deployed on both In-SITU SSDs and the Host machine. The configuration of the In-Situ SSD will be detailed here since the deployment of these modules on Host machines are similar to the installation of services in operating system. Before deploying the proposed method on the SSD-CSDs it is necessary to understand the firmware functions of the CSDs. The firmware can be represented as logical layers each representing a specific functionality right from managing the flash blocks for data read and write Input and output operations to supporting and environment like running a container or virtual machine on CSDs. The logical layers utilizing the underlying hardware of CSDs can be divided into three fundaments parts: - The base layer will be responsible for managing, storing the data in the flash blocks. There will be a layer named flash translation layer which helps in identifying or resolving the physical address in flash blocks from the logical address shared by the Host. Flash management

will be another logical layer which manages the wear and tear level of the hardware, garbage collection and optimizing the storage hardware. The top most layer will be the host manager or host driver which create an environment inside the storage or CSDs which will allow to host the container or virtual machine and at the same time communication can be established between Host and CSDs through this layer. The system presented here uses all these layers for embedding the proposed method to authenticate CSD and Host further a secured communication channel between them.

[0023] The above mentioned seven components perform specific functions with respect to the securing and encrypting the communication between host and CSDs. On CSDs these software components are deployed inside the container using different processors to keep the implementation load limited on different CSDs resource. To make the implementation lightweight Micropython or C programming can be used to deploy the functions on the existing logical layers of the CSDs without much change. The hardware layers in the CSDs and the embedded location of the software components are shown in the Fig 2. The tapping of the CSDs hardware resources for the seven components are discussed

below: PCIe NVMe Protocols: These protocols are used for high-speed data transfer between the host and the SSD. The Communication component of the algorithm, which handles the sending and receiving of tokens between the host and the CSDs, would be deployed here.

[0024] Computational Storage Processors: These processors within the CSDs perform computations directly on the CSDs, reducing data movement and improving performance. The Nonce Generator Token Generator, and Session Token Generator components of the algorithm that run on the CSD could be deployed here. These components involve computations such as generating random nonces and performing hash and encryption operations. Co-Processor HW Accelerator: This hardware unit is designed to speed up certain types of computations. It works in conjunction with the main CPU to improve overall system performance. Depending on the specific system design and the computational requirements of the algorithm, some components of the algorithm (such as the hash and encryption operations) could potentially be offloaded to this co-processor to improve performance. Real-Time Processors, ECC, AES: These components are used for real-time processing and encryption. The Token verifier

and Session Key Generator components of the algorithm, which involve hash and encryption operations, could potentially be deployed on these processors. In addition, the ECC (Error Correction Code) could be used to detect and correct errors in the data being transmitted between the host and the CSD.

[0025] **Results**: To test the performance of the developed method, a simulated environment was built. The environment consisted of: A computer with an i3 processor, 2 GB of memory, and an SSD was used as the storage system. Two host systems were connected to this storage system within a simulated network. To mimic limited resource environments, virtual machines (VMs) were created on the storage system. Each VM had just 1 virtual CPU (vCPU) and 1 GB of memory, simulating a CSDs resources with limited processing power and memory. The developed method was installed on all three systems (two hosts and the storage system). Memory Saturation: During testing, the VM's memory was intentionally Tilled to its capacity (saturated). This forced the system to rely on the virtual SSD (SSD accessed through the VM), mimicking real-world disk access scenarios. Large applications were deployed on all three systems to simulate real-world usage. The test aimed to measure how much processing

power (CPU utilization) the method required while running alongside existing operating system services and other default applications. A Java Virtual Machine (JVM) and a mobility Remote Procedure Call (RPC) package were installed on all systems. This enabled remote execution of applications or functions, another aspect of the method being tested. To demonstrate the method's efficiency (lightweight nature), three performance parameters were monitored during the testbed execution.

[0026] Real Time (Wall Clock Time): This refers to the actual elapsed time from when a process starts to its completion. It includes all time spent, regardless of the process's state (running, idle, waiting). This is essentially the time you'd see on a wall clock. User Time: This metric represents the total CPU time spent executing the process's user-mode code. Think of it as the active processing time dedicated to the specific tasks of your program (excluding interactions with the operating system kernel). Time spent waiting for resources (like I/O operations) or being interrupted by other processes doesn't contribute to user time. System Time: This metric reflects the CPU time spent executing the process's code within the kernel. This includes time spent on system calls, which are requests made by

the program to the operating system for specific services. Unlike user time, system time doesn't include time spent executing library code (functions that your program uses but are not part of its core logic).

[0027] The graph Fig. 4 depicts the program's execution time at a virtual storage node. The analysis of the values indicates an average system time of approximately 12 milliseconds (0.011789 seconds) for program execution. This low system time suggests that the proposed method has minimal execution overhead. The negligible execution time further implies low time complexity for the algorithm. In simpler terms, the time required to execute the program does not significantly increase with the size of the input data. This characteristic translates to a reduction in the computational power needed by the system, making the algorithm a lightweight mechanism suitable for deployment on smart devices with resource constraints.

[0028] Storage or Memory Consumption at Virtual storage Node: The system utilizes AES and MD5 hashing, chosen for their low storage (ROM) and execution (RAM) requirements compared to asymmetric algorithms. This reduces performance overhead for resource-constrained smart devices. Fig 5

shows execution memory or storage usage of the system. AES Encryption Evaluation at virtual storage Node: The system employs AES for encryption and decryption, including the SecureAuthKey algorithm for session token generation. AES operates on a plaintext message with a secret key (EK) and random nonces (S and R) to produce ciphertext. Decryption retrieves the original message by reversing this process. Fig 6 illustrates the encryption time required for text, image, and audio Tiles on resource-constrained devices. As expected, the text exhibits the fastest encryption time due to its smaller data size compared to images and audio. [FIG. 6]. AES Decryption Evaluation at virtual storage Node: FIG 7. depicts the decryption time required for various data types (text, image, audio) on resource-constrained devices. As expected, smaller data sizes like text decrypt faster compared to images and audio.

[0029] The above results evaluate the efficiency and effectiveness of the SafeConnectKey Method. Analysis of system execution time indicates low processing overhead on the node. Additionally, Storage, RAM and ROM usage are considered to assess computational overhead. The results demonstrate that the SafeConnectKey Method successfully achieves: Lightweight Mechanism: Low

resource consumption for efficient operation on smart devices. Security Algorithm: Provides robust security measures for data protection. Scalability: Able to handle increasing data volumes or user numbers effectively. Secure and Flexible Framework: Offers a secure foundation that can be adapted for various applications.

[0030] It will be appreciated that variations of the above-disclosed and other features and functions, or alternatives thereof, may be desirably combined into many other different systems or applications. Also, that various presently unforeseen or unanticipated alternatives, modifications, variations or improvements therein may be subsequently made by those skilled in the art which are also intended to be encompassed by the following claims.

COMPLETE SPECIFICATION
APPLICANT (S): VELLORE INSTITUTE OF TECHNOLOGY

5. CLAIMS

I/We Claim

1. A method and system for secure communication between computational storage drives and host system, comprising steps of:

a nonce generator for generating random nonces (S and R) at CSD and Host wherein a random nonce (S) is generated at CSD using which an authentication token “AuthTd” and a session token “SessTd” is created and the provided keys and sent to the Host wherein the host upon receiving the tokens, extracts the nonce (S) and generates another authentication token “AuthTd'” wherein the “AuthTd'” is matched with “AuthTd” and host generates a nonce (R) and a session key “SessK” and sends a new session token “SessTh” to the CSD wherein the CSD extracts the nonce (R) from the received session token and generates a session key “SessKd” wherein the “SessKd” is matched with “SessK” in order to establish a secured communication between the CSD and the Host system.

2. The method as claimed in claim 1 wherein a hash function calculates the authentication tokens (AuthTd, AuthTd') and the session key (SessK, SessK')

using a one-way hashing algorithm wherein the hashing ensures data integrity and prevents tampering wherein it takes inputs (Host ID, authentication key, nonces) and performs a mathematical operation to generate a unique and fixed-size output (hash value).

3. The method as claimed in claim 1 wherein an encryption engine encrypts the nonce (S) sent by the Host to create a session token (SessTd) and encrypts the CSD's nonce (R) to create a confirmation session token (SessTh) wherein the encryption protects sensitive data during transmission and uses a shared encryption key (EnK) and a chosen encryption algorithm (like AES) to transform the nonce into an unreadable format.

4. The method as claimed in claim 1 wherein a decryption engine decrypts the received session tokens (SessTd, SessTh) to retrieve the original nonces wherein the decryption engine uses the shared encryption key (EnK) and the decryption algorithm to convert the encrypted session tokens back into their original nonce values.

5. The method as claimed in claim 1 wherein an authentication module handles authentication by generating and verifying authentication tokens (AuthTd,

AuthTd') wherein it combines Host ID, authentication key (AuthK), and nonces, and feeds them to the hash function to create AuthTd (at Host) and AuthTd' (at CSD) wherein the authentication module compares them for a successful authentication check.

6. The method as claimed in claim 1 wherein a session management module manages session key generation and utilizes the session key for secure communication wherein it combines various inputs (like Host ID, received tokens, retrieved nonces) and feeds them to the hash function to generate the session key (SessK, SessK') and verifies if the locally generated and calculated session keys match, indicating a successful session establishment.

7.The method as claimed in claim 1 wherein a communication interface enables data exchange between the Host and the CSD which is not directly involved in the secure communication algorithm but crucial for transmitting tokens and establishing the communication channel wherein the communication interface handles sending and receiving of authentication tokens (AuthTd, AuthTd') and session tokens (SessTd, SessTh) between the Host and the CSD.

6. SIGNATURE WITH DATE: 26/08/2024

HARIPRASAD KS

PATENT AGENT (IN/PA-3636)

COMPLETE SPECIFICATION
APPLICANT (S): VELLORE INSTITUTE OF TECHNOLOGY

7. ABSTRACT

A nonce generator for generating random nonces (S and R) at CSD and Host wherein a random nonce (S) is generated at CSD using which an authentication token “AuthTd” and a session token “SessTd” is created and the provided keys and sent to the Host wherein the host upon receiving the tokens, extracts the nonce (S) and generates another authentication token “AuthTd'” wherein the “AuthTd'” is matched with “AuthTd” and host generates a nonce (R) and a session key “SessK” and sends a new session token “SessTh” to the CSD. Further, The CSD extracts the nonce (R) from the received session token and generates a session key “SessKd” wherein the “SessKd” is matched with “SessK” in order to establish a secured communication between the CSD and the Host system. **[FIG. 1]**